

Explicación del Proyecto de PLN

Proyecto Final - Procesamiento de Lenguaje Natural

Repositorio

El código fuente del proyecto está disponible en: https://github.com/lDavidSantiago/PLN_Final_Project

Idea general

El proyecto es un analizador de comandos escritos en español. Recibe frases como **crea una carpeta llamada proyectos** o **mueve los archivos pdf a respaldo**, las divide en partes, valida su estructura y obtiene una representación semántica parecida a un comando de consola. No ejecuta acciones reales; solo interpreta y simula el resultado.

Flujo del sistema

El procesamiento se realiza en etapas claras:

1. El usuario escribe una frase.
2. El **lexer** normaliza el texto y produce tokens.
3. El **parser** revisa si los tokens cumplen la gramática.
4. Si la frase es válida, se construye un árbol de derivación.
5. El análisis semántico extrae acción, objeto, nombre, extensión y destino.
6. El detector de ambigüedad revisa frases con más de una interpretación.
7. El simulador muestra el comando equivalente, por ejemplo **mkdir proyectos**.

Archivos principales

Archivo	Función
main.py	Menú principal; conecta lexer, parser, semántica, ambigüedad y simulador.
lexer.py	Normaliza la frase y convierte cada palabra en tokens como VERBS, DET, TYPE, EXT, PREP o NAMING.
dfa.py	Implementa el autómata finito determinista usado para formalizar el análisis léxico.
grammar.py	Define la gramática libre de contexto: variables, terminales, producciones y símbolo inicial.
parser.py y tree.py	Validan la secuencia de tokens y representan el árbol de derivación.
semantic.py	Convierte el árbol en una estructura con significado.
ambiguity.py	Detecta frases ambiguas, por ejemplo cuando falta una preposición clara.
simulator.py	Genera una simulación del comando final: mkdir , mv , rm , cp o ls .

Lexer y DFA

El **lexer** primero pasa la frase a minúsculas, elimina signos de puntuación y separa el texto por palabras. Luego clasifica cada palabra según un vocabulario controlado. Por ejemplo:

```
crea una carpeta llamada proyectos
VERBS DET TYPE NAMED_AS NAMING
```

El DFA se describe formalmente como $(Q, \Sigma, \delta, q_0, F)$. En el proyecto sirve para representar que una palabra reconocida lleva a un estado final según su categoría: verbo, determinante, tipo de objeto, extensión, preposición, conector o nombre.

Gramática y parser

Después de tokenizar, el parser usa una gramática libre de contexto $G = (V, T, P, S)$. Las reglas más importantes son:

```
S -> CMD | CMD_COMP
CMD_COMP -> CMD CONNECTOR CMD
CMD -> VERBS OBJ | VERBS OBJ DEST
OBJ -> DET TYPE | DET TYPE EXT
OBJ -> DET TYPE NAMED_AS NAMING
DEST -> PREP NAMING | NAMING
```

Estas reglas permiten aceptar comandos simples y compuestos. Por ejemplo, **crea una carpeta llamada datos y mueve los archivos csv ahí** se analiza como dos comandos unidos por un conector.

Análisis semántico y simulación

El módulo semántico extrae los datos importantes del árbol. Además, traduce verbos normalizados a acciones internas:

crear	mover	eliminar	copiar	renombrar	listar
mkdir	mv	rm	cp	mv	ls

Ejemplo completo:

```
Entrada: mueve los archivos pdf a respaldo
Tokens: VERBS DET TYPE EXT PREP NAMING
```

```
Semantica:
action: mv
verb: mover
object_type: archivo
extension: pdf
destination: respaldo
```

```
Simulacion:
mv *.pdf respaldo/
```

Ambigüedad

El lenguaje natural puede ser ambiguo. El proyecto detecta casos como:

```
elimina los archivos de documentos
```

Esta frase puede significar eliminar archivos dentro de **documentos** o eliminar archivos llamados/asociados con **documentos**. También se detectan destinos implícitos, como **mueve los archivos pdf documentos**, porque **documentos** podría ser nombre o destino. Si hay ambigüedad, el sistema pide seleccionar una interpretación antes de simular el comando.